

Online Quiz System: A Java OOP Implementation

Hanwen Zhang (1308159) & Zimo Qin — CPS 2231 W02

Wenzhou-Kean University

LOGO HERE

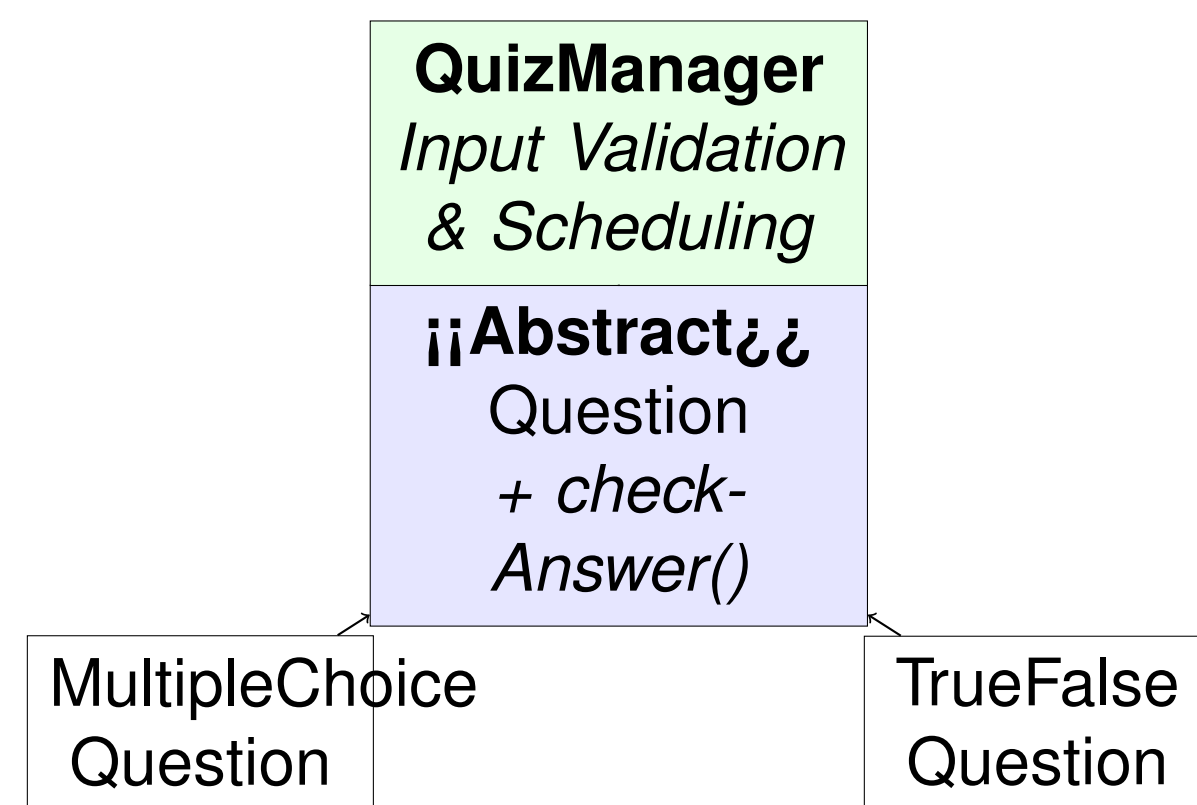
1. System Overview

This project presents a robust **Online Quiz System** built using Java Object-Oriented Programming (OOP) principles. The system encapsulates question display, answer verification, and input validation into a modular architecture.

- **Core Philosophy:** Designed using abstract classes, inheritance, and method overriding to ensure extensibility.
- **Key Functionality:** Manages the full quiz process, including input detection, score calculation, and result display.

2. System Architecture

The application follows a hierarchical class structure centered around the abstract `Question` class.



Components:

- **Inheritance Tree:** `MultipleChoice` and `TrueFalse` inherit from `Question`.
- **Management:** `QuizManager` schedules the interaction between `Student` entities and question logic.

3. Implementation Details

The **Abstract Question Class** serves as the blueprint for all question types.

Polymorphism in Action:

- **Multiple Choice:** Overrides `display()` to automatically generate A/B/C/D options using character offsets.
- **True/False:** Implements strict logic where "A" maps to `True` and "B" maps to `False`.

Code Snippet: Polymorphism

```
// Implementation in TrueFalseQuestion
@Override public boolean checkAnswer(String answer) // Validates A/B input logic return
answer.equalsIgnoreCase(correctAnswer);
```

4. Workflow Diagram

The system ensures valid data entry through a cyclical check within `QuizManager`.

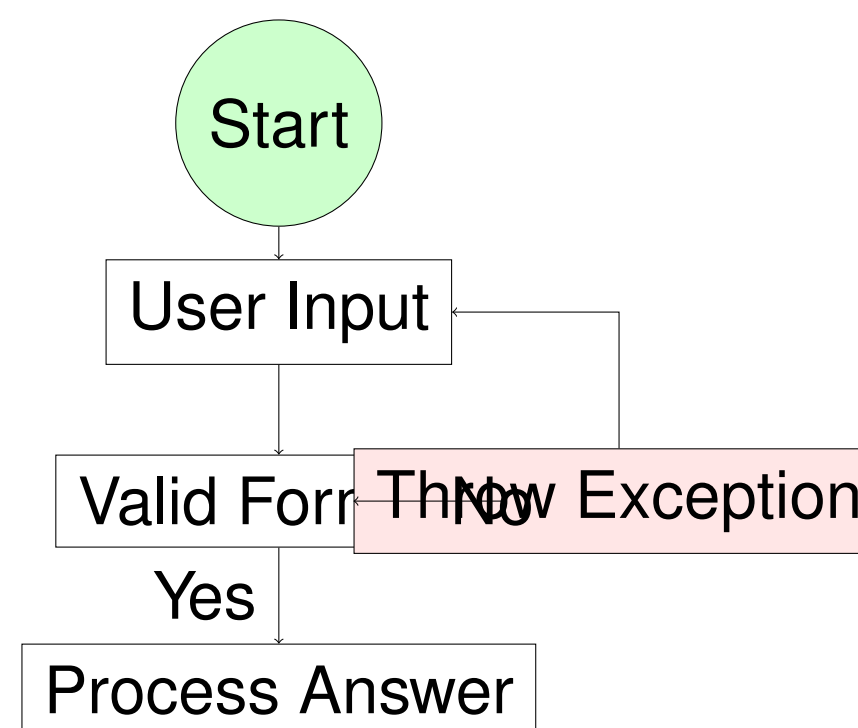


Figure 2: The logic flow for handling student input and exceptions.

5. Robustness & Validation

A critical feature is the `validateInput` mechanism within the `QuizManager`.

- **Exception Handling:** Uses `InvalidInputException` to handle erroneous user data.
- **Empty Input:** Rejects empty strings immediately to prevent null pointers.
- **Type Checking:** For True/False questions, inputs other than "A" or "B" (case-insensitive) trigger a retry loop.
- **Thread Safety:** The `Student` class uses the `synchronized` keyword for `addScore`, enabling future expansion to multi-threaded environments.

6. Conclusion

We successfully implemented an OOP-based Quiz System that prioritizes modularity and error resilience.

Key Achievements:

- **Reliability:** Secured via custom exception handling.
- **Scalability:** Abstract design allows easy addition of new question types (e.g., Fill-in-Blank) without breaking existing code.
- **Maintainability:** Clear separation of logic (`Question`), data (`Student`), and control (`QuizManager`).

References

- Horstmann, C. S. (2018). *Core Java Volume I—Fundamentals*. Prentice Hall.
- Oracle Documentation. (2023). *Java Platform, Standard Edition Java Development Kit Version 17 API Specification*.